Reinforcement Learning (RL)

Author: Refat Othman

PART 1 — What is Reinforcement Learning?

> **Reinforcement Learning**: given a set of rewards or punishments, learn what actions to take in the future.

1 Intuition First (No Math Yet)

Ask students:

> How does a child learn to walk?

- Try → Fall → Pain (negative reward)
- Try → Stand → Smile (positive reward)
- Over time → Learns best actions

That is Reinforcement Learning.

Definition

Reinforcement Learning is:

> Learning by interacting with an environment using rewards and punishments.

Unlike supervised learning:

- ✕ No labeled dataset
- ✕ No correct answers given
- ☑ Agent discovers good behavior through trial and error

RL vs Other Learning Types

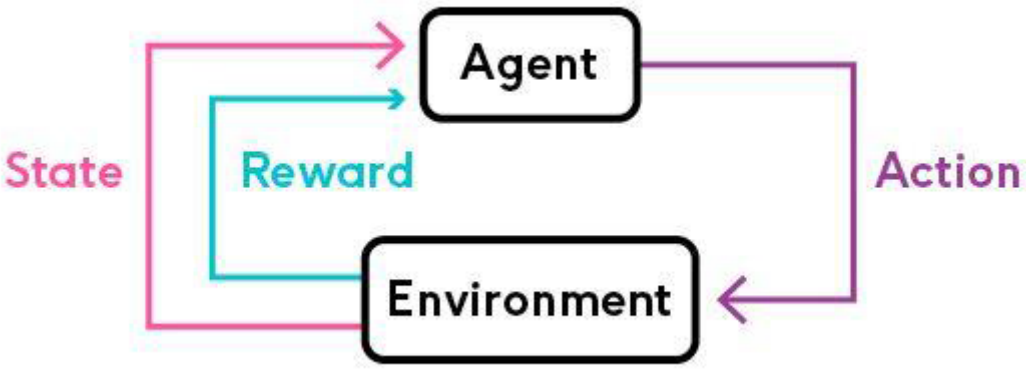
Type	Data	Goal
Supervised	Labeled data	Predict output
Unsupervised	No labels	Find patterns
Reinforcement	Rewards	Learn best actions

2 Agent–Environment Model

(Reference concept from your slides: Agent → Action → Environment → Reward → New State)

In RL we have:

- **Agent** → the learner
- **Environment** → where agent acts
- **State (s)** → current situation
- **Action (a)** → decision taken
- **Reward (r)** → feedback



Loop:

State → Action → Reward → New State → Repeat

---

3 Markov Decision Process (MDP)

A **Markov Decision Process (MDP)** is a mathematical model used to describe decision-making problems where outcomes are partly random and partly controlled by an agent.

It provides the formal foundation of Reinforcement Learning.

An MDP consists of:

- **Set of states (S)** → all possible situations the agent can be in
- **Set of actions (A)** → all possible decisions the agent can take
- **Transition model P(s' | s, a)** → probability of moving to next state s' after taking action a in state s
- **Reward function R(s, a, s')** → immediate feedback received after transition

The Markov Property

> The future depends only on the current state (not the full history).

This means: If we know the current state, we have all the information needed to decide what happens next.

Simple Example: Robot in a Grid

Imagine a robot inside a 3×3 grid:

S . . . . G

- States → Each cell position (0,0), (0,1), ...
- Actions → {up, down, left, right}

- Transition → If robot moves right from (0,0), it goes to (0,1)
- Reward → +10 if it reaches Goal (G), 0 otherwise

Now suppose the robot is at state (0,1).

To decide the next move, we do NOT care how it arrived there. We only care that it is currently in (0,1).

That is the Markov property.

Real-Life Example: Self-Driving Car

State:

- Current speed
- Distance to car ahead
- Traffic light color

Action:

- Accelerate
- Brake
- Turn

Reward:

- +1 for safe driving
- -100 for collision

The car decides what to do based only on the current situation — not the full history of the trip.

Why MDP is Important

Reinforcement Learning algorithms (like Q-learning) assume the problem can be modeled as an MDP.

Without MDP structure:

- We cannot properly define states
- We cannot compute expected future rewards
- Learning becomes unstable

PART 2 — Q-Learning

Now we move from intuition to algorithm.

> **Q-learning** is a method for learning a function Q(s, a), which estimates the value of performing action a in state s.

It is called a **model-free** Reinforcement Learning algorithm because it does NOT need to know the transition probabilities P(s'|s,a). Instead, it learns directly from experience.

What is Q(s, a)?

Q(s, a) answers the question:

"If I am in state s and I choose action a, how much total reward can I expect in the future?"

Important:

- It includes the **immediate reward**
- It also includes **future rewards** (discounted by γ)

Q(s, a) = "How good is it to take action a in state s?"

If Q is high → good action. If Q is low → bad action.

Key Idea for Students

The agent does NOT learn directly which action is best. It learns the VALUE of each action.

Then it chooses the action with the highest value.

So learning happens in two steps:

1. Estimate Q-values
2. Choose action with highest Q-value

Simple Numerical Example

Imagine we are at state (0,0) in a grid.

Suppose after training we have:

Q((0,0), 'right') = 5.2 Q((0,0), 'down') = 3.1 Q((0,0), 'left') = -1.0 Q((0,0), 'up') = 0.5

What does this mean?

If the agent goes RIGHT from (0,0), it expects to collect about 5.2 total future reward.

So the best action is:

max Q(s,a) → 'right'

This is how policy emerges from Q-values.

Q-Learning Algorithm

Initialize:

Q(s, a) = 0 for all states and actions

Update rule:

$Q(s, a) \leftarrow Q(s, a) + \alpha[(r + \gamma \max Q(s', a')) - Q(s, a)]$

Explain Each Term

- α (alpha) → learning rate (how fast we update)
- γ (gamma) → discount factor (importance of future rewards)
- r → reward received
- max Q(s', a') → best possible future value

Interpretation:

New Q = Old Q + LearningRate × (Better Estimate − Old Q)

Explore vs Exploit

One of the biggest challenges in Reinforcement Learning is the **exploration–exploitation tradeoff**.

Exploitation

Exploitation means:

→ Choosing the action that currently has the highest Q-value.

Advantage:

- Maximizes reward based on what the agent already knows.

Problem:

- The agent might get stuck choosing a "locally good" action.
- It may never discover a better long-term strategy.

Exploration

Exploration means:

→ Trying random or less-known actions to gather more information.

Advantage:

- The agent may discover better actions.

Problem:

- It may choose bad actions and receive low rewards.

Why Is This Important?

Imagine a student choosing restaurants:

- If they always go to their favorite restaurant (exploit), they may never discover a better one.
- If they always try new restaurants (explore), they may waste money on bad ones.

A balance is needed.

Solution → ε-greedy Strategy

ε-greedy is a simple and practical strategy used in Q-learning to balance exploration and exploitation.

What It Is

ε (epsilon) is a small number between 0 and 1 that represents the probability of exploring.

The decision rule works as follows:

- With probability (1 − ε): choose the action with the highest Q-value (exploit what we know).
- With probability ε: choose a random action (explore new possibilities).

So the agent mostly behaves intelligently, but sometimes acts randomly to gather more information.

Why We Use It

If we NEVER explore:

- The agent may get stuck with a suboptimal policy.
- It may think an action is bad just because it tried it too early.

If we ALWAYS explore:

- The agent never stabilizes.
- Learning becomes random and inefficient.

ε-greedy gives a simple balance:

- Enough exploration to discover better strategies.
- Enough exploitation to maximize reward.

Intuition Example

Imagine at the beginning of training all Q-values are 0. The agent does not know which action is best.

If ε = 0:

- It will always choose the first action (since all Q-values are equal).
- It may never try other actions.

If ε = 0.2:

- It will try random actions 20% of the time.
- It gradually learns which direction leads to higher reward.

Over time, good actions get larger Q-values. The agent naturally starts exploiting more often.

Practical Strategy: ε-Decay

In real applications, we often start with high exploration:

ε = 1.0 → 100% exploration at beginning

Then gradually reduce it during training:

ε → 0.1 or smaller

Why?

- Early stage → agent needs to explore the environment.
- Later stage → agent should act more confidently and exploit what it learned.

This makes learning stable and efficient.

Numerical Example

Suppose in state (0,0) we have:

Q(right) = 5.0 Q(down) = 4.8 Q(left) = 1.2 Q(up) = 0.5

The best action is "right".

If ε = 0.2:

- 80% of the time → choose "right"
- 20% of the time → randomly choose among {up, down, left, right}

This means sometimes the agent may try "down" and discover that in the long run it actually leads to higher total reward.

Important Notes

At the beginning of training:

- We usually use larger ε (more exploration).

Later in training:

- We reduce ε (more exploitation).

This is called **ε-decay**.

Example:

ε = 1.0 → Start fully exploring ε gradually decreases → 0.1

So the agent explores first, then becomes more confident and exploits what it learned.

PART 3 — Coding From Scratch

We build a simple Grid World.

Grid:

S . . . X . . . G

S = Start G = Goal (+10 reward) X = Obstacle (-10 reward)

Step 1 — Create Environment

```
import numpy as np
import random

# Grid size
rows, cols = 3, 3

# Rewards
rewards = np.zeros((rows, cols))
rewards[2, 2] = 10    # Goal
rewards[1, 1] = -10   # Obstacle

# Actions: up, down, left, right
actions = ['up', 'down', 'left', 'right']
```

Explanation Line-by-Line

- numpy → to create grid
- rewards → matrix storing reward for each cell
- actions → possible movements

Step 2 — Initialize Q-table

```
Q = {}

for r in range(rows):
    for c in range(cols):
        Q[(r, c)] = {a: 0 for a in actions}
```

Explanation:

- Q is a dictionary
- Each state (r, c) has 4 actions
- All values start at 0

Step 3 — Hyperparameters

Hyperparameters are values that we choose BEFORE training starts. They control how the learning process behaves. They are not learned automatically — we set them manually.

```
alpha = 0.1   # learning rate
gamma = 0.9   # discount factor
epsilon = 0.2 # exploration rate
episodes = 500
```

Describe Each One (What & Why)

1 alpha (Learning Rate)

What it is:

- Controls how much we update Q-values each time.
- Determines how fast the agent learns.

Why it matters:

- If alpha is too large → learning becomes unstable.
- If alpha is too small → learning becomes very slow.

Example: If alpha = 1 → agent completely replaces old Q-value with new estimate. If alpha = 0.01 → agent updates very slowly.

---

2 gamma (Discount Factor)

What it is:

- Controls how much we care about future rewards.
- Value between 0 and 1.

Why it matters:

- If gamma = 0 → agent only cares about immediate reward.
- If gamma close to 1 → agent cares about long-term rewards.

Example: If reaching goal requires 5 steps:

- Small gamma → agent may not plan ahead.
- Large gamma → agent prefers long-term strategy.

---

3 epsilon (Exploration Rate)

What it is:

- Probability of choosing a random action.
- Controls exploration vs exploitation.

Why it matters:

- High epsilon → more exploration.
- Low epsilon → more exploitation.

Example: If epsilon = 0.5 → agent explores 50% of time. If epsilon = 0 → agent never explores.

---

4 episodes

What it is:

- Number of times the agent plays the game from start to finish.

Why it matters:

- More episodes → more experience → better learning.
- Too few episodes → agent does not learn good policy.

Example: If episodes = 10 → learning will likely be poor. If episodes = 1000 → learning becomes stable.

Step 4 — Training Loop

```
def move(state, action):
    r, c = state
    if action == 'up': r = max(r-1, 0)
    if action == 'down': r = min(r+1, rows-1)
    if action == 'left': c = max(c-1, 0)
    if action == 'right': c = min(c+1, cols-1)
    return (r, c)

for episode in range(episodes):
    state = (0, 0)

    while state != (2, 2):
        # ε-greedy
        if random.uniform(0,1) < epsilon:
            action = random.choice(actions)
        else:
```

```
            action = max(Q[state], key=Q[state].get)

        next_state = move(state, action)
        reward = rewards[next_state]

        # Q update
        best_future = max(Q[next_state].values())
        Q[state][action] += alpha * (
            reward + gamma * best_future - Q[state][action]
        )

        state = next_state
```

After Training — Show Learned Policy

```
for state in Q:
    print(state, max(Q[state], key=Q[state].get))
```

Students will observe arrows pointing toward goal.

Notes

1. Change gamma to 0 → No future planning
2. Change epsilon to 0 → No exploration
3. Reduce episodes → Poor learning

Visual Explanation to Draw on Board

Draw:

State (0,0)

Q-table example:

Action	Q-value
Up	0.2
Down	1.3
Left	-0.1
Right	2.8

Best action → Right

Final Concept — Function Approximation (Advanced Teaser)

Problem:

If state space is huge → Q-table impossible.

Solution:

Approximate Q(s,a) using a neural network.

This leads to:

Deep Q-Network (DQN)

Mention only — no deep math.

Where is RL used in real life?

Examples:

- Robotics
- Self-driving cars
- Game playing (Chess, Go)
- Recommendation systems
- Finance trading